American University of Sharjah
College of Engineering
Computer Science & Engineering Department
Introduction to Computer Systems – COE 251L
Instructor: Ms. Donthi Sankalpa

Lab #5: Indirect Addressing modes

Important Addressing Operators

In x86 assembly language, various operators are used to simplify memory addressing, data access, and manipulation. These operators provide useful ways to reference the size, length, and type of data in memory, making it easier to write and understand assembly code. Some of the most commonly used operators include PTR, TYPE, LENGTHOF, and SIZEOF. These operators help programmers work with data types and memory more efficiently.

Operator	Description	Example
PTR	Used to explicitly define the type of a variable during an operation, particularly for memory access.	MOV AL, BYTE PTR [BX] ; Accesses a byte at the address in BX
TYPE	Returns the size (in bytes) of the data type of a specified variable.	MOV AX, TYPE myWord ; If myWord is a WORD, AX = 2
LENGTHOF	Returns the number of elements in an array.	MOV CX, LENGTHOF myArray ; If myArray has 5 elements, CX = 5
SIZEOF	Returns the total size (in bytes) of an array or variable.	MOV DX, SIZEOF myArray ; If myArray has 5 words, DX = 10 (5 * 2 bytes)

Indirect Addressing Modes:

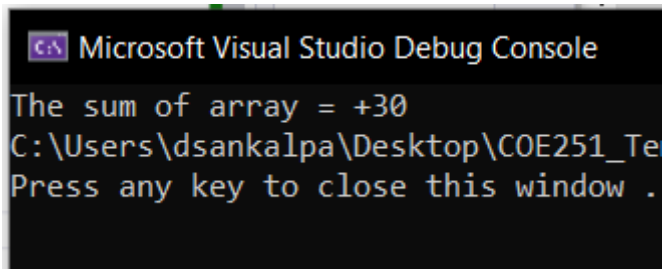
In x86 assembly, indirect addressing modes provide a way to reference memory locations using registers, allowing for more flexible data access. By using registers like EBX, EBP, ESI, and EDI, you can store addresses or offsets that point to data in memory. For example, EBX and EBP can be used to access memory in the data or stack segments (SS for EBP), while ESI and EDI are commonly used for string and array processing. Indirect addressing is particularly useful for manipulating arrays, pointers, and stack frames. Below is a table detailing different addressing modes, combinations of calls, and examples.

Addressing Mode	Description	Example
Immediate	Uses a constant value directly.	MOV EAX, 5 ; Move the value 5 into EAX
Register	Uses the contents of a register.	MOV EAX, EBX ; Move the value in EBX into EAX
Direct Memory	Accesses a specific memory location.	MOV AL, [myVar] ; Move the byte at myVar into AL
Direct-Offset	Accesses memory at an offset from a base address.	MOV AL, [myArray + 2] ; Access the third element in myArray

Indirect	Uses the value in a register as a memory address. Can use registers like EBX, EBP, ESI, and EDI.	MOV EAX, [EBX] ; Access the memory location pointed to by EBX
Indexed	Combines a base address with an index register. Index registers can be EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP.	MOV AL, [arrayB + ESI] ; Accesses an element in arrayB using ESI as an index

Lab Work (Questions 1-5):

1. Given an array of 5 words: `myArray WORD 2, 4, 6, 8, 10`, write an assembly program to find the sum of all elements in the array using indexed addressing.

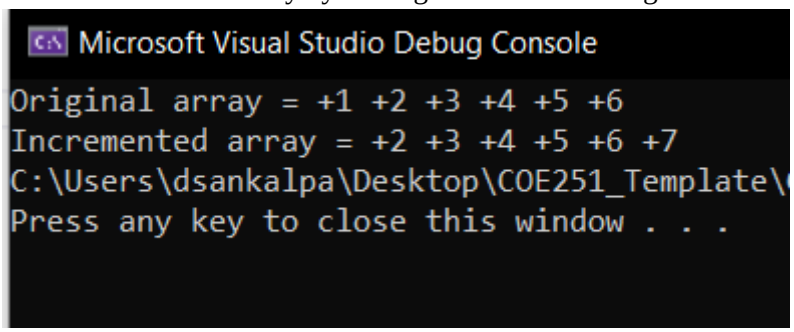


Screenshot:

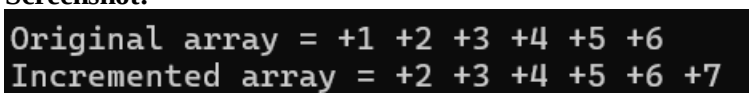
Solution:

```
TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data
        str1 byte "The sum of the array = ", 0
        myArray word 2, 4, 6, 8, 10

.code
main PROC
        mov edx, offset str1
        call writestring

        mov esi, 0
        mov bx, 0
        mov ecx, sizeof myArray

        again:
        add bx, [myArray + esi]
        add esi, 2
        loop again

        movzx eax, bx
        call writeint

        exit
main ENDP
END main
```

2. Given an array of 6 bytes: `myArray BYTE 1, 2, 3, 4, 5, 6`, write an assembly program to increment each element of the array by 1 using indirect addressing.



Screenshot:

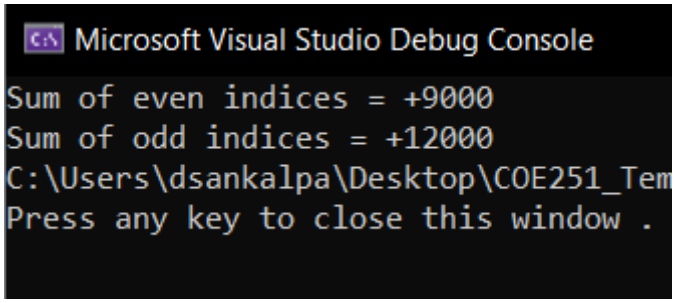


Solution:

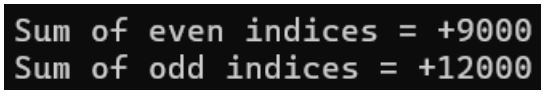
```
TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data
        str1 byte "Original array = ", 0
        str2 byte "Incremented array = ", 0
        str3 byte " ", 0
        myArray byte 1, 2, 3, 4, 5, 6

.code
main PROC
        mov edx, offset str1
        call writestring

        mov eax, 0
        mov esi, offset myArray
        mov edx, offset str3

        mov ecx, sizeof myArray

        again1:
        mov al, [esi]
        inc esi
        call writeint
        call writestring
        loop again1

        call crlf
```

```
        mov edx, offset str2
        call writestring

        mov esi, offset myArray

        mov edx, offset str3

        mov ecx, sizeof myArray

        again2:
        mov al, [esi]
        inc al
        inc esi
        call writeint
        call writestring
        loop again2

        exit
main ENDP
END main
```

3. Given an array of 6 double words: myArray DWORD 1000, 2000, 3000, 4000, 5000, 6000, write an assembly program to calculate the sum of elements at even indices (0, 2, 4) and the sum of elements at odd indices (1, 3, 5). Store the sum of even indices in EAX and the sum of odd indices in EBX. Use a loop to iterate through the array.



Screenshot:

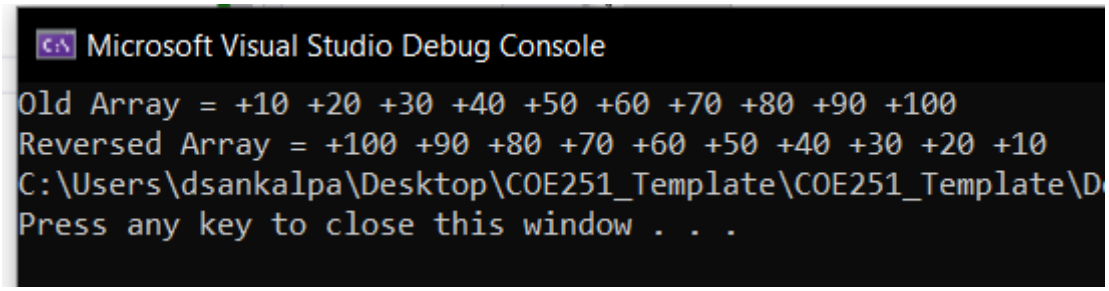


Solution:

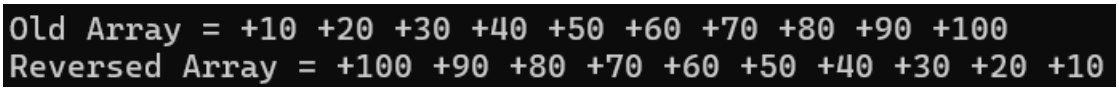
```
TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data
        str1 byte "Sum of even indices = ", 0
        str2 byte "Sum of odd indices = ", 0
        sum1 dword 0
```

```
        sum2 dword 0
        myArray dword 1000, 2000, 3000, 4000, 5000, 6000

.code
main PROC
        mov edx, offset str1
        call writestring

        mov eax, 0
        mov ecx, 3
        mov esi, offset myArray

        again1:
        mov eax, [esi]
        add sum1, eax
        add esi, 8
        loop again1

        mov eax, sum1
        call writeint

        call crlf

        mov edx, offset str2
        call writestring

        mov ebx, 0
        mov ecx, 3
        mov esi, offset myArray
        add esi, 4

        again2:
        mov ebx, [esi]
        add sum2, ebx
        add esi, 8
        loop again2

        mov ebx, sum2
        mov eax, ebx
        call writeint

        exit
main ENDP
END main
```

4. Given a source array of 10 bytes: sourceArray BYTE 10, 20, 30, 40, 50, 60, 70, 80, 90, 100 and an empty destination array: destArray BYTE 10 DUP(0), write an assembly program to copy each element from sourceArray to destArray in the reverse order.



Screenshot:



Solution:

```
TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data
	str1 byte "Old Array = ", 0
	str2 byte "Reversed Array = ", 0
	str3 byte " ", 0
	sourceArray byte 10, 20, 30, 40, 50, 60, 70, 80, 90, 100
	destArray byte 10 DUP (0)

.code
main PROC
	mov edx, offset str1
	call writestring

	mov edx, offset str3
	mov esi, offset sourceArray

	mov ecx, sizeof sourceArray

	again1:
	movzx eax, byte ptr [esi]
	call writeint
	call writestring
	inc esi
	loop again1

	call crlf

	mov ecx, sizeof sourceArray
```

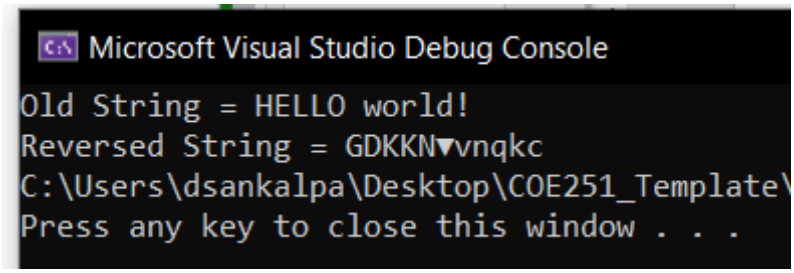
```
        mov esi, 9
        mov eax, 0
        mov ebx, 0

        again2:
        mov al, byte ptr [sourceArray + ebx]
        mov [destArray + esi], al
        dec esi
        inc ebx
        loop again2

        mov edx, offset str2
        call writestring

        mov ebx, offset destArray
        mov ecx, sizeof sourceArray

        mov edx, offset str3

        again3:
        movzx eax, byte ptr [ebx]
        call writeint
        call writestring
        inc ebx
        loop again3

        exit
main ENDP
END main
```

My Code

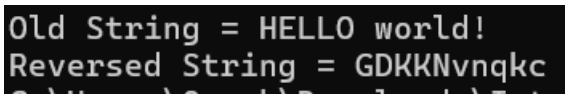
```
TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data
        sourceArr byte 10,20,30,40,50,60,70,80,90,100
        destArr byte 10 DUP (0)
        str1 byte "Old Array = ", 0
        str2 byte "Reversed Array = ", 0
        str3 byte " ", 0

.code
main PROC
        mov eax, 0
        mov ebx, 0
```

```
    mov edx, offset str1
    call writestring

    mov esi, offset sourceArr
    mov edx, offset str3

    mov ecx, lengthof sourceArr

    first:
    mov al, [esi]
    call writeint
    call writestring
    inc esi
    loop first

    call crlf

    mov edx, offset str2
    call writestring

    mov edi, offset destArr
    mov edx, offset str3

    mov ecx, lengthof destArr

    second:
    dec esi
    mov bl, [esi]
    mov [edi], bl
    mov al, [edi]
    call writeint
    call writestring
    inc edi
    loop second

    exit
main ENDP
END main
```

5. Given a null-terminated string: myString BYTE "HELLO world!", 0, write an assembly program that "rotates" each character in the string to the left in the ASCII table. For example ('B' becomes 'A', 'C' becomes 'B', ..., 'A' becomes '@', 'b' becomes 'a', etc. The program should modify the string in place and handle uppercase, lowercase, and special characters.



Screenshot:



Solution:

```
TITLE
Name/Purpose
Here

INCLUDE
Irvine32.inc

.data
	str1 byte "Old
String = ", 0
	str2 byte
"Reversed String =
", 0
	myString byte
"HELLO world!",
0

.code
main PROC
	mov edx,
offset str1
	call
writestring

	mov edx,
offset myString
	call
writestring

	call crlf

	mov edx,
offset str2
	call
writestring
```

```
        mov eax, 0

        mov esi, 0
        mov ecx, sizeof
myString

        again:
        dec byte ptr
[myString + esi]
        inc esi
        loop again

        mov edx,
offset myString
        call
writestring

        exit
main ENDP
END main
```

My Code

```
TITLE
Name/Purpose
Here

INCLUDE
Irvine32.inc

.data
        str1 byte "Old
String = ", 0
        str2 byte
"Reversed String =
", 0
        myString byte
"HELLO world!",
0

.code
main PROC
        mov edx,
offset str1
        call
writestring
```

```
        mov edx,
offset myString
        call
writestring

        call crlf

        mov edx,
offset str2
        call
writestring

        mov esi, 0
        mov edx,
offset myString

        mov ecx,
lengthof myString

        again:
        dec byte ptr
[edx+esi]
        inc esi
        loop again

        call
writestring

        exit
main ENDP
END main
```